

Queue

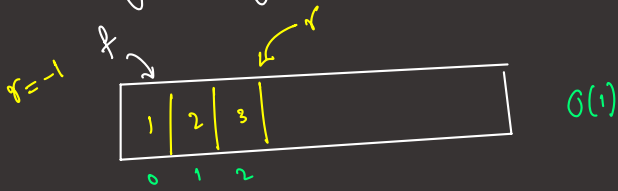
first in first out

insertion order = deletion order

- insert/add/enqueue $O(1)$
- deletion/remove/dequeue $O(1)$ DS Deque
- peek() $\rightarrow$ Front $O(1)$

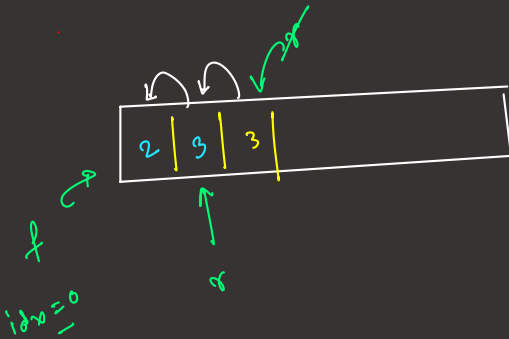
Implementation of Queue

(i) using Array



insert
[1, 2, 3]

remove



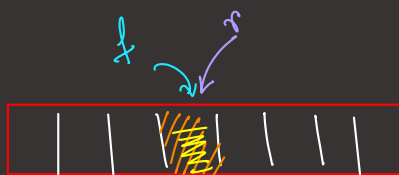
full $\rightarrow \{ r - f = \text{size} - 1 \}$



$(r+1) \% \text{size} = f$

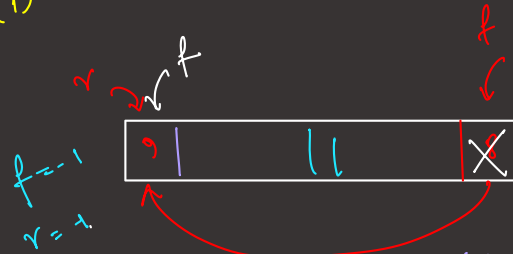
condition of empty

$\{ f = -1, r = -1 \}$



remove

$O(1) \rightarrow$ Circular Queue



add(1)

add(2)

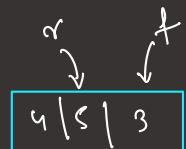
add(3)

remove() $\rightarrow 1$

add(9)

$r_{\text{rear}} = (r_{\text{rear}} + 1) \% \text{size}$

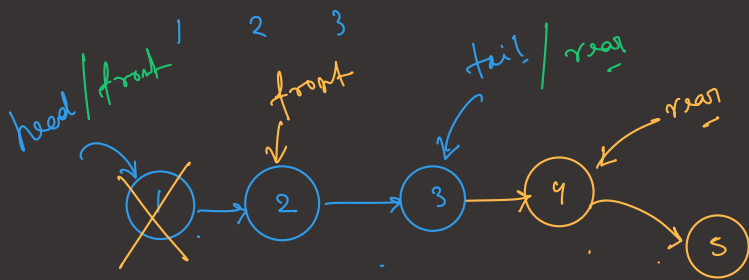
$f_{\text{front}} = (f_{\text{front}} + 1) \% \text{size}$



Queue using Linked List

$O(1)$ remove()
 $O(1)$ peek()

add() $O(1)$



Queue Using JCF

Queue $\leftarrow$ interface.

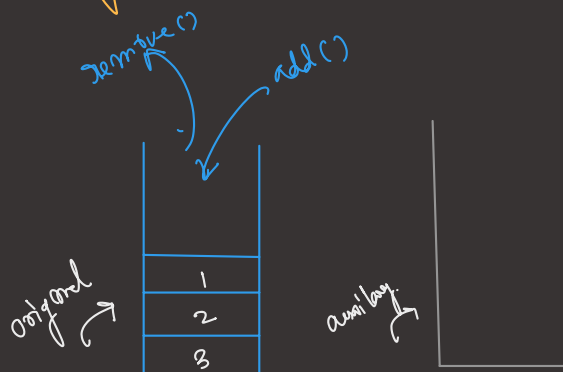
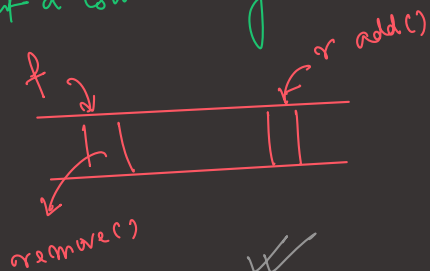
✓ Queue < Integer > q₁ = new LinkedList < > ();

Queue < Integer > q₂ = new ArrayDeque < > ();

① = ③

Question

implement a Queue using two stacks

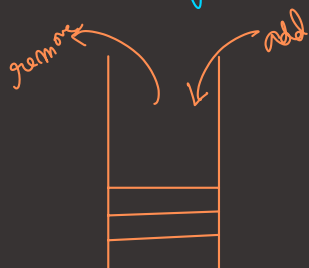


H.W

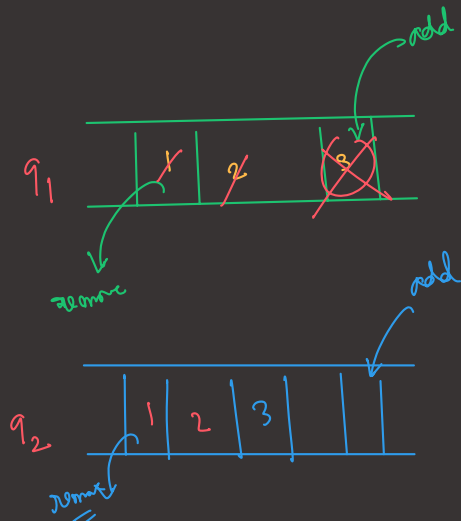
way 1
 add $\rightarrow O(1)$
 remove $\rightarrow O(n)$

or way 2
 add $\rightarrow O(n)$
 remove $\rightarrow O(1)$

Stack Using two Queue



push
 [1 2 3]
 pop [3 2 1]



way 1
 push $\rightarrow O(1)$
 pop $\rightarrow O(n)$

way 2
 push $\rightarrow O(n)$
 pop $\rightarrow O(1)$

Pop() ? top = -1

```
if (!q1.isEmpty()) {
    while (!q1.isEmpty()) {
        top = q1.remove();
    }
}
```

```
if (q1.isEmpty()) {
    break;
}
```

```
q2.add(top);
} else {
```

```
while (!q2.isEmpty()) {
    top = q2.remove();
    if (q2.isEmpty()) {
        break;
    }
    p1.add(top);
}
```

```
return top;
```

$$O(n)$$

Pop, peek

Question

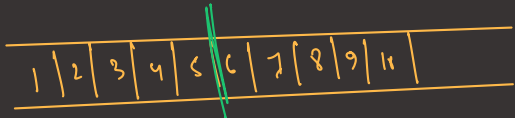
Interleave 2 halves of a Queue. (even length)

Given: 1 2 3 4 5 | 6 7 8 9 10

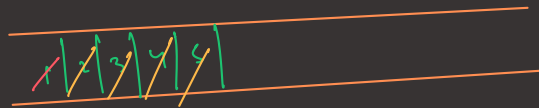
O/P: 1 6 2 7 3 8 4 9 5 10

upto $q.size()/2$ from initial queue remove and put it into the new queue

initial q

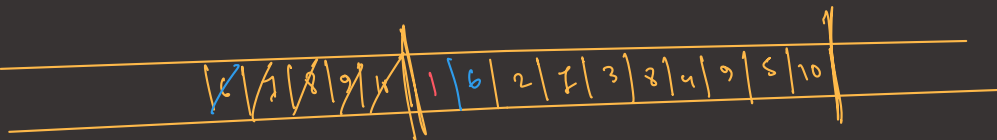


new q



```
while (!newq.isEmpty()) {
    initialq.add(newq.remove());
    initialq.add(initialq.remove());
}
```

initial



Question

First Non-repeating Letter in a Stream of character

→ all character will be in small letter

Stream = aabccxb

✓ a

o/p
a

a a

-1

a a b

b

a a b c

b

a a b c c

b

a a b c c x

b

a a b c c x b

x

queue < char > q

for (int i = 0 to str.length) {

ch = str.charAt(i)

q.add(ch), map update

q.peek() →

front

map[front] → frequency > 1

q.remove()

ch [freq] = 1 → ch → first non-repeating letter

① freq[1] = [26] 'a' → 'z'

q → empty() return = -1

② Queue < character > q

③ for (i = 0 to str.length) {

ch = str.charAt(i)

q.add(ch)

freq['ch' - 'a']++

q.remove() → till I found 1st non-repeating letter

freq map [26] =

2	2	2	...	1	
a	b	c		x	z

map → 'ch' - 'a'

a	a
---	---

freq[a] = 2

b	c	x	b
---	---	---	---

peek → freq[x] = 1

freq = 1 2

o/p = a

= -1

= b

= b

= b

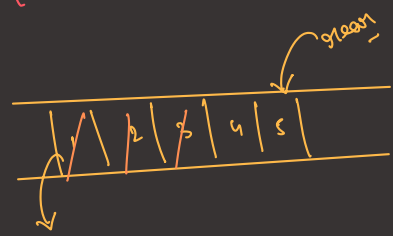
= b

= x

Question

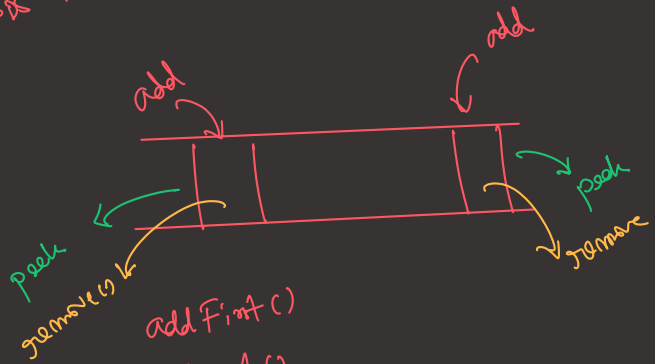
Queue Reversed

Queue $\rightarrow [1, 2, 3, 4, 5]$, print in reverse order
 o/p : 5 4 3 2 1



operation $\rightarrow$ remove $\rightarrow O(1)$
Deque $\leftrightarrow$ Double ended queue

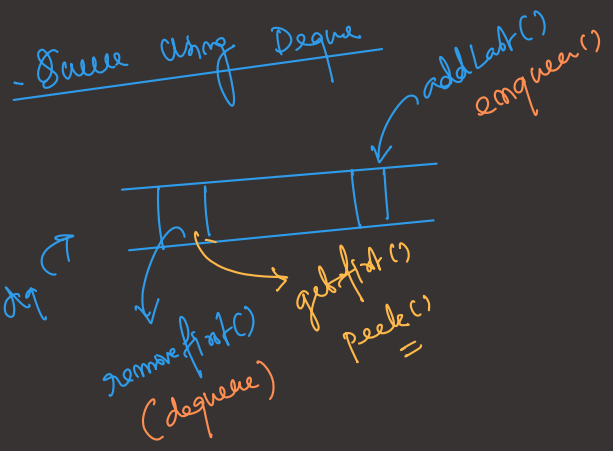
Deque



- addFirst()
- addLast()
- removeFirst()
- removeLast()
- getFirst()
- getLast()

```
Deque<Integer> dq = new LinkedList<>();
```

Queue using Deque



Stack using Deque

